Database and Database Users

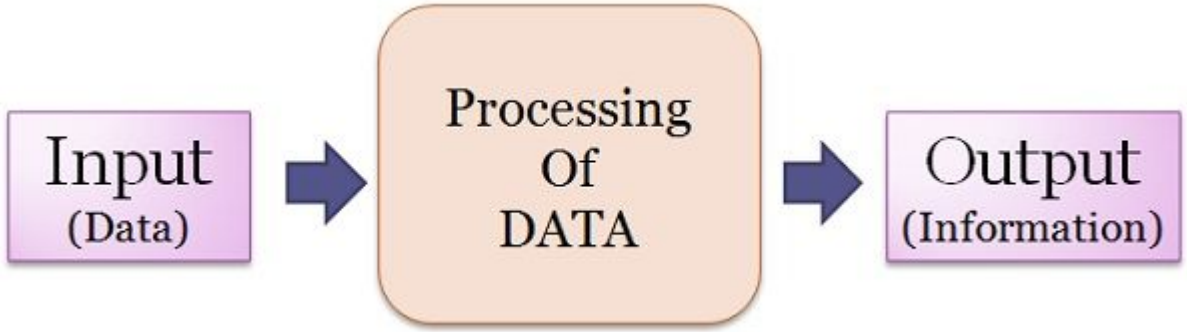
Unit 1

Data

- Data are the raw facts that can be obtained after some experiments or observations. Raw data is of no use until and unless we process it to find some useful information form it.

Information

- When that data is analyzed, structured, and given context to make it useful, we find information.



Database

- A database is the collection of related persistent data and contains information relevant to an enterprise. The database is also called the repository or container for a collection of data files. For example, university database for maintaining information about students, courses and grades in university.
- A database is **an organized collection of structured information, or data, typically stored electronically in a computer system**.

Data: Raw, unorganized facts and figures that have no inherent meaning on their own. In a DBMS, this is what is stored in tables (rows and columns) before any logic is applied.

- *Examples:* 10204, Smith, 25.50, 2026-05-05.

Information: Data that has been processed, structured, or presented in a specific context to make it meaningful and useful for decision-making.

- *Examples:* "Employee ID 10204 belongs to Smith,"

OR

- "The average daily sales for May is $25.50."

Database Management System (DBMS)

- Management of data involves a way to store data and also provides a mechanism for manipulation of that data. Database management systems are basically designed to manage large volume of information.
- A database system is basically just a computerized record-keeping system.
- A database system involves four major components: **data, hardware, software, and users.**
- The database management system (DBMS) is the software that handles all access to the database. It is defined as the collection of interrelated data and a set of programs to access those data.
- The primary goal of DBMS is to store and retrieve data in both convenient (easy method) and efficient (capable of performing well) manner. Some of the examples of DBMS are Oracle, SQL-Server, MySQL, MS Access etc.

Applications of DBMS

- **Banking:** To store information about customers, their account number, balance etc.
- **Airlines:** For reservations and schedule information.
- **Telecommunication:** To keep records of customers, call made, balance left, generating monthly bills etc.
- **Universities:** To keep records of students, courses, marks of students etc.
- **Sales:** To keep information of customers, products list, purchase information etc.
- **Manufacturing:** To store orders, tracking production of items etc.
- **Human Resources:** To keep records of employee, their salary, bonus etc.

1. Self describing nature of database system:

- The **self-describing nature of a DBMS** means that the database system doesn't just store data, it also stores **information about the data often called as metadata**.

The **System Catalog** (often called the Data Dictionary) acts as a central repository for metadata. It includes:

- **Structure Definitions:** Tables, columns, and data types (e.g., INT, VARCHAR).
- **Constraints:** Primary keys, foreign keys, and "NOT NULL" requirements.
- **Security:** Information about user permissions and access rights.
- **Storage Mappings:** Where the data is physically located on the disk.

- The knowledge stored within the catalog is named meta-data, and it describes the structure of the first database. The catalog is employed by the DBMS software and also by database users such as database administrators who required to know the information about the database structure.

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
….	….	…..
….	….	…..
….	….	…..
Prerequisite_number	XXXXNNNN	PREREQUISITE

2. Insulation between programs and data :

- But in the case of DBMS, database and the application program are separately situated. The structure of data files is stored in the DBMS catalog separately from the access programs. Hence in most cases DBMS access programs do not need such changes. We call this property as program-data independence.
- Program Data Independence: Suppose you need to add a new column for "Middle Name" to a Users table. In a DBMS, existing applications that only ask for "First Name" and "Last Name" continue to work perfectly. They don't even know the third column exists because the DBMS handles the mapping between the stored record and what the program requested.
- Physical Data Independence:You decide to move your database files from a local hard drive to a high-speed Cloud storage, or you change the file organization from a sequential scan to a B-tree index. Because the application only interacts with the *logical* table name, it remains unaware of where the bytes are physically located. You only update the internal mapping in the catalog.
-

3. Support of multiple view of the data:

- Let us consider an example of student information system of a college, where all the data of student, courses, information of college etc. are stored in a database. It is obvious that there is more than one user of this system and also their interest is different. i.e. student are generally interested in finding out the marks obtained, whereas teachers are able to put marks, view information about students, similarly, visitors are able to find the information of the college.
- A database typically has many users, and their perspective of viewing the database is different. In the example given above, every user is accessing data from the same database but what they see and can access is different. A view may be subset of the database or it may contain virtual data that is derived from the database files (stored query) but is not stored explicitly. A view can present data that doesn't exist in the tables but is calculated on the fly, such as an "Age" column derived from a "Date of Birth" field.

(a)

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

(b)

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

Figure 1.5
Two views derived from the database in Figure 1.2. (a) The TRANSCRIPT view. (b) The COURSE_PREREQUISITES view.

4. Sharing of data and multi user transaction processing:

- Database system should allow multiple users to access same database at the same time. For example, in online air ticket reservation system many users are accessing the same site at the same time. Hence for every seat, DBMS should ensure that only one user should be given access to reserve the seat.
- A transaction is an executing program or process that includes one or more database accesses, such as reading or updating of database records.
- Hence to ensure the correctness of this type of transaction DBMS must include concurrency control software to ensure that several users trying to update same data do so in a controlled manner so that the result of the updates is correct. In order to do so, DBMS uses lock based protocol and time stamp based protocol.

Actors on the scene

- Many persons are involved in the design, use, and maintenance of a large database with a few hundred users.
- Here we will consider people who may be called "Actors on the Scene", whose jobs involve the day-to-day use of a large database.

i. Database Administrators:

They create user accounts, control access, and ensure data security by authorizing only trusted users. DBAs are also responsible for handling security breaches and optimizing system performance.

i) **Authorizing Access:** They act as the gatekeepers, creating user accounts and assigning permissions (e.g., who can read vs. who can delete data).

ii)**Coordinating and Monitoring:** They ensure that the database is running smoothly for multiple users.

iii)**Resource Management:** They are responsible for the physical environment—deciding when to upgrade the server's RAM or purchase more disk space for growing data.

iv)**Backup and Recovery:** A critical DBA task is ensuring that if the system crashes, the data can be restored with zero loss.

ii. Database Designers:

- Database Designers are responsible :
  - for identifying the data to be stored in the database
  - for choosing appropriate structures to represent and store this data.
- Their role is focused on the logical and physical design of the data before the database is even populated.
- Database designer typically interact with each potential group and users and develop a view of the database that meets the data and processing requirements of this groups.

Requirement Analysis: They talk to the end-users to understand what data is actually needed (e.g., "Do we need to store the student's middle name or just an initial?").

Defining the Structure: They choose the tables, columns, and data types. They decide which field will be the **Primary Key**.

Relationship Mapping: They define how different tables relate to one another (e.g., how a 'Student' table links to a 'Course' table).

Normalization: They ensure the design is efficient and free of redundancy to prevent data anomalies.

iii. End Users:

End users are the people whose jobs require access to the database for querying, updating and generating reports; the database primarily exists for their use.

- There are several categories of end users:
  - a. **Casual End User:**
    - These users access the database occasionally, but they may need different information each time.
    - **Interaction Style:** They use sophisticated database query languages (like SQL) or menu-based interfaces to browse the data.
    - **Expertise:** They are typically middle-to-upper-level managers or occasional browsers who know what they want but don't necessarily perform the same task every day.
    - **Example:**

Scenario: The Dean needs to know: "Which third-year Computer Science students have a GPA above 3.8, have completed 'Operating Systems', but have NOT yet enrolled in 'Database Systems' for the upcoming semester?"
The Menu Problem: There is no button on the clerk's screen for this specific, multi-layered question.
The SQL Solution: Because the Dean (or their assistant) understands a **sophisticated query language**, they can write a single, custom command to join the Students, Grades, and Enrollment tables to get the answer immediately.

b. Naive or Parametric end user

- Parametric End Users are the unsophisticated who lacks DBMS knowledge but they frequently use the database applications in their daily life to get the desired results

Example:

Bank Tellers: They enter an account number and an amount to process a deposit.

Airline Reservation Clerks: They enter a destination and date to check for available seats.

c.Sophisticated End User:

- These are high-level users who have a deep understanding of the DBMS and its capabilities.
- **Interaction Style:** They often write their own complex queries or even develop their own software tools to interact with the database for analysis or research.
- **Expertise:** They include engineers, scientists, and business analysts who use the database to solve complex problems or perform data mining.

Example:

A scientist using a database to analyze weather patterns over 50 years.

An AI engineer pulling data from a vector database to train a RAG-based legal assistant.

- **Stand-alone users:**
  Standalone Users represent a specific category of individuals who manage their own data independently of a large organizational system.
- **Standalone users are defined by the following characteristics:**
  - These users maintain personal databases by using ready-to-use program packages that provide easy-to-use, menu-based, or graphics-based interfaces.
  - **Purpose:** They typically use these systems for personal tasks rather than organizational data processing.
  - **Tooling:** An example of a standalone user is someone who maintains a personal address book or a collection of recipes using a specialized software package that manages the data in the background

Personal Finance Manager: An individual using a software package like Quicken or a specialized personal budget app to track their income, expenses, and bank balances.
Contact Management: A person maintaining an extensive digital address book or a list of professional networking contacts using a personal information manager (PIM).

Note: Even though they work alone, they are considered "Actors on the Scene" because they perform all the roles themselves—they are effectively their own **DBA**, **Designer**, and **End User** for their personal data

- **System Analysts and Application Programmers (Software Engineers):**
  System analysts determines the requirements of end users, especially naive and parametric end users, and develop specifications for transactions that meet these requirements.
- *Application Programmers* implement these specifications as programs; then they test, debug, document, and maintain these canned transactions. Such analyst and programmers are called *Software Engineers*.

Actors on the Scene focus on the **content** and the **application** (the specific data within the database).

Workers Behind the Scene focus on the **DBMS software product** (the engine that makes database management possible).

In an academic context, if you are building an application for a library, you are an **Actor on the Scene**. If you are working for a company like Oracle or Microsoft to write the code for the database engine itself, you are a **Worker Behind the Scene**.

Workers behind the scene

- In addition to those who design, use, and administer a database, others are associated with the design, development, and operation of the DBMS software and system environment. These persons are typically not interested in the database content itself. They provide the tools and environment that the actors use. We call them the workers behind the scene, and they include the following categories:
- **DBMS system designers and implementers** design and implement the DBMS modules and interfaces as a software package. A DBMS is a very complex software system that consists of many components, or modules, including modules for implementing the catalog, query language processing, interface processing, accessing and buffering data, controlling concurrency, and handling data recovery and security. The DBMS must interface with other system software such as the operating system and compilers for various programming languages.
- **Tool developers** These individuals design and implement software packages that sit "on top" of the DBMS. They create specialized tools for database modeling, performance monitoring, graphical user interfaces (GUIs), and application development environments.
- **Operators and maintenance personnel** (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system. . They ensure the servers are running, the network is functional, and the software is updated.
- Although these categories of workers behind the scene are instrumental in making the database system available to end users, they typically do not use the database contents for their own purposes.

1. Controlling Redundancy

- In traditional file processing, every user group maintains its own files for handling its data processing applications. For example, consider the UNIVERSITY database, in the traditional approach, each group independently keeps files on students. The accounting office keeps data on registration and related billing information, whereas the registration office keeps track of student courses and grades. Other groups may further duplicate some or all of the same data in their own files.
- This redundancy in storing the same data multiple times leads to several problems. First, there is the need to perform a single logical update—such as entering data on a new student—multiple times: once for each file where student data is recorded. This leads to duplication of effort. Second, storage space is wasted when the same data is stored repeatedly, and this problem may be serious for large databases. Third, files that represent the same data may become inconsistent. For example, one user group may enter a student's birth date erroneously as 'JAN191988', whereas the other user groups may enter the correct value of 'JAN-29-1988'.
- In the database approach, the views of different user groups are integrated during database design. Ideally, we should have a database design that stores each logical data item—such as a student's name or birth date—in only one place in the database. This is known as data normalization, and it ensures consistency and saves storage space.

2. Restricting unauthorized access

- When multiple users share a large database, it is likely that most users will not be authorized to access all information in the database.
- For example, financial data is often considered confidential, and only authorized persons are allowed to access such data.
- In addition, some users may only be permitted to retrieve data, whereas others are allowed to retrieve and update.
- Hence, the type of access operation retrieval or update must also be controlled. Typically, users or user groups are given account numbers protected by passwords, which they can use to gain access to the database.
- DBMS should provide a security and authorization subsystem, which the DBA uses to create accounts and to specify account restrictions.
- Then, the DBMS should enforce these restrictions automatically.
- Notice that we can apply similar controls to the DBMS software.
- For example, only the DBA's staff may be allowed to use certain privileged software, such as the software for creating new accounts. Similarly, parametric users may be allowed to access the database only through the canned transactions developed for their use.

3. Providing Persistent Storage for Program Objects

- Databases can be used to provide persistent storage for program objects and data structures.
- Programming languages typically have complex data structures, such as record types in Pascal or class definitions in C++ or Java. The values of program variables or objects are discarded once a program terminates, unless the programmer explicitly stores them in permanent files, which often involves converting these complex structures into a format suitable for file storage. When the need arises to read this data once more, the programmer must convert from the file format to the program variable or object structure.
- Object-oriented database systems are compatible with programming languages such as C++ and Java, and the DBMS software automatically performs any necessary conversions. Hence, a complex object in C++ can be stored permanently in an object-oriented DBMS. Such an object is said to be persistent, since it survives the termination of program execution and can later be directly retrieved by another C++ program.
- The persistent storage of program objects and data structures is an important function of database systems. Traditional database systems often suffered from the so-called impedance mismatch problem, since the data structures provided by the DBMS were incompatible with the programming language's data structures.
- Object-oriented database systems typically offer data structure compatibility with one or more object-oriented programming languages.

4. Providing Storage Structures and Search Techniques for Efficient Query Processing

- Database systems must provide capabilities for efficiently executing queries and updates.
- Because the database is typically stored on disk, the DBMS must provide specialized data structures and search techniques to speed up disk search for the desired records
- In order to process the database records needed by a particular query, those records must be copied from disk to main memory. Therefore, the DBMS often has a buffering or caching module that maintains parts of the database in main memory buffers.

5. Providing Backup and Recovery

- A DBMS must provide facilities for recovering from hardware or software failures.
- The backup and recovery subsystem of the DBMS is responsible for recovery.
- For example, if the computer system fails in the middle of a complex update transaction, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the transaction started executing.
- Alternatively, the recovery subsystem could ensure that the transaction is resumed from the point at which it was interrupted so that its full effect is recorded in the database.
- Disk backup is also necessary in case of a catastrophic disk failure.

6. Providing Multiple User Interfaces

- Because many types of users with varying levels of technical knowledge use a data-base, a DBMS should provide a variety of user interfaces.
- These include query languages for casual users, programming language interfaces for application programmers, forms and command codes for parametric users, and menu-driven interfaces and natural language interfaces for standalone users.
- Both forms-style interfaces and menu-driven interfaces are commonly known as graphical user interfaces (GUIs).

7. Representing Complex Relationships among Data

- A database may include numerous varieties of data that are interrelated in many ways.
- A DBMS must have the capability to represent a variety of complex relationships among the data, to define new relationships as they arise, and to retrieve and update related data easily and efficiently.

8. Enforcing Integrity Constraints

- Most database applications have certain integrity constraints that must hold for the data. A DBMS should provide capabilities for defining and enforcing these constraints.
- The simplest type of integrity constraint involves specifying a data type for each data item. For example, the value of Name must be a string of no more than 30 alphabetic characters.